



Dr. Vishwanath Karad
MIT WORLD PEACE
UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

FACULTY OF SCIENCE
SCHOOL OF COMPUTER SCIENCE
2024-2025

A
PROJECT REPORT ON
“Gesture-to-Text Conversion
using Deep Learning”

By

Koustubh Deodhar	1132230197
Amaan Sayyed	1132230434
Anna Martin	1132230156
Adnan Pathan	1132230164

IN PARTIAL FULFILLMENT OF
MASTER OF COMPUTER APPLICATION (SCIENCE)
DR. VISHWANATH KARAD MIT WORLD PEACE
UNIVERSITY



DEPARTMENT OF COMPUTER SCIENCE AND APPLICATIONS

Certificate

This is to certify that We, Koustubh Deodhar, Amaan Sayyed, Anna Martin, Adnan Pathan student of MCA-Science Semester IV has successfully completed in fulfilment of MCA- Science under Dr. Vishwanath Karad MIT World Peace University, for the academic year 2023-2024.

Internal Guide

Dr. Jalindar R. Gandal
Head-MCA- Science

Dr. Anuradha Kannade
Program Director

Prof. Dr. Sarmishtha Desai
Associate Dean Academics

Prof. Dr. Shubhalaxmi Joshi
Asso. Dean, Faculty Affairs

Prof. Dr. Mangesh Badekar
Dean, School of CSE

Date: ____ / ____ / ____
External Examiners:

1. _____ 2. _____



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

ACKNOWLEDGEMENT

We wish to express our gratitude for providing the facilities of the Institute and for her encouragement during the course of this work. We would also like to gratefully acknowledge the enthusiastic supervision of my internship guide Assistant Professor Apurva Sharma, School of Computer Science, MIT-WPU, Pune for her continuous, valuable guidance, patience, constant care and kind encouragement throughout the internship work that made me present this internship report in an efficient manner.

Koustubh Deodhar

Amaan Sayyed

Anna Martin

Adnan Pathan

MCA (Science)



Dr. Vishwanath Karad

**MIT WORLD PEACE
UNIVERSITY** | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

DECLARATION

I hereby declare that the project work entitled “Gesture-to-Text Conversion using Deep Learning” submitted to the MIT-WPU, Pune is a record of an original work done by us under the guidance of Assistant Professor Apurva Sharma and this project work is submitted in the partial fulfilment successfully completed in fulfilment of MCA- Science under Dr. Vishwanath Karad MIT World Peace University, for the academic year 2023-2024.

Koustubh Deodhar

Amaan Sayyed

Anna Martin

Adnan Pathan

MCA (Science)

INDEX

Sr. No.	Contents	Page No.
Chapter 1	INTRODUCTION	
	1.1 Existing System	
	1.2 Problem Definition- Need of Computerization	
Chapter 2	PROPOSED SYSTEM	
	2.1 Proposed System	
	2.2 Objectives of System	
	2.3 User Requirements	
	2.4 Operating Environment – Hardware and Software	
Chapter 3	ANALYSIS AND DESIGN	
	Module List	
	3.1 ERD, UML Diagram	
	3.2 Table Design (if applicable)	
	3.3 Data Dictionary (if applicable)	
	3.4 Screen Shots	
	3.5 Reports (if applicable)	
	3.6 Test Procedures and Implementation (if applicable)	
Chapter 4	USER MANUAL	
	4.1 User Manual	
	4.2 Menu Explanation	
Chapter 5	CONCLUSIONS	
	5.1 Limitations & Drawbacks	
	5.2 Future Enhancement	
	5.3 Conclusions	
	5.4 References & Bibliography	

Existing System

- Traditional input methods like keyboards and touchscreens are widely used for text input and interaction with digital systems. However, these conventional devices have limitations in terms of flexibility and accessibility, especially in environments where physical contact is impractical or impossible. For instance:
- Physical Limitations: Traditional devices may not be suitable for users with disabilities or in situations requiring hands-free operation.
- Limited Interaction Options: These methods do not provide intuitive or natural ways for users to interact with technology.
- Restricted Accessibility: In dynamic environments like public spaces, healthcare, or gaming, relying solely on physical input devices can be inefficient and cumbersome.
- The lack of an alternative input system for these scenarios leads to challenges in user interaction, slowing down workflows and hindering accessibility. Existing solutions do not leverage modern advancements in gesture recognition and deep learning, which could offer innovative and intuitive methods for text input.

1.1 Problem Definition-Need of Computerization

Conventional input methods often lack the adaptability and accessibility needed in modern, dynamic environments. They prove inefficient in hands-free scenarios, challenging for individuals with disabilities, and limiting in sectors like healthcare, gaming, or public spaces where natural and intuitive interactions are essential for productivity and user experience.

Need for Computerization

- A gesture-based system leveraging deep learning overcomes the limitations of traditional input methods by enabling intuitive, hands-free interactions through air-written gesture recognition. This approach enhances accessibility, facilitates real-time input, and ensures seamless compatibility across devices. Its versatility makes it particularly suitable for domains such as education, gaming, healthcare, and public systems. By moving beyond conventional methods, such systems drive innovation, improve efficiency, and promote inclusivity in human-computer interaction.

2.1 Proposed System:

- The proposed system revolutionizes user interaction by introducing a gesture-to-text conversion platform powered by advanced deep learning technologies. It offers a hands-free, intuitive alternative to traditional input devices like keyboards and touchscreens, making it ideal for dynamic environments or users with accessibility challenges. The system employs Convolutional Neural Networks (CNNs) to detect gestures and Long Short-Term Memory (LSTM) networks to process sequential data, ensuring accurate text conversion.
- Using cameras or motion sensors, the system captures air-written gestures and translates them into text in real time, providing instant feedback to users. Its user-friendly interface ensures accessibility for all, while cross-platform compatibility supports seamless operation on desktops, laptops, and mobile devices. Additionally, its scalable and adaptable design allows integration with IoT devices, smart home systems, and public kiosks, broadening its applications.
- This system addresses the need for innovative input methods in fields like healthcare, education, gaming, and public services. By enhancing accessibility, boosting productivity, and offering real-time interaction, it transforms the way users engage with technology.

2.2 Objectives of System:

- The Gesture-to-Text Conversion System aims to provide an accessible and intuitive input method by converting air-written gestures into text, offering a practical alternative to traditional input devices like keyboards.
- The system's primary objective is to create a robust and accurate gesture recognition model that detects and converts hand gestures into text in real time, using Convolutional Neural Networks (CNN) to achieve reliable, high-precision results.
- Additionally, cross-platform compatibility is a key goal, enabling the system to function seamlessly on Windows, Mac, and mobile devices so that users can interact with the technology across their preferred platforms.
- To make the system widely accessible, a user-friendly interface is prioritized, allowing individuals with varying levels of technical expertise to use the system effortlessly.
- The intuitive design minimizes the learning curve, making it suitable for a broad audience. Overall, this project seeks not only to improve user interaction through alternative input methods but also to showcase the potential of deep learning in advancing gesture recognition technology, highlighting new and innovative ways for users to engage with digital devices.

2.3 User Requirements:

Hardware Requirements:

1. **Camera or Motion Sensor:** The system requires a camera or sensor capable of capturing hand gestures with sufficient resolution and frame rate for accurate recognition.
2. **Processing Power:** A processor with adequate speed and memory is necessary to handle real-time gesture recognition, especially if running on mobile devices.
3. **Platform Compatibility:** The system should be compatible with hardware across Windows, Mac, and mobile devices, requiring different versions of the software for each platform.

Software and Frameworks

4. **Programming Language:** The primary language for development is Python due to its compatibility with machine learning frameworks and libraries.
5. **Deep Learning Frameworks:** The system relies on TensorFlow and Keras for building and training the Convolutional Neural Network (CNN) model required for gesture recognition.
6. **Image Processing Library:** OpenCV is required for capturing and processing video or images from the camera, enabling gesture detection.
7. **Numerical Computation:** NumPy is used to support numerical operations needed during data manipulation and model training.
8. **Visualization Tools:** Matplotlib or similar tools are used to visualize data and model performance during the development phase.

Model Architecture

1. **Convolutional Neural Network (CNN):** A CNN is required to process image frames and identify air-written gestures with high accuracy.
2. **Recurrent Neural Network (LSTM):** An LSTM layer is used to handle the time-series aspect of gestures, maintaining context from one frame to the next, which is essential for interpreting sequential hand movements.
3. **Fully Connected Layers (Dense Layers):** Dense layers are required for the final prediction of gestures, based on features learned from the CNN and LSTM layers.

Data Requirements

Training Dataset:

A diverse dataset of air-written characters is necessary to train the model effectively, with a mix of real-world and synthetic data to account for various conditions and styles of gestures. **Data Augmentation:** Data augmentation techniques may be used to increase the dataset's robustness, ensuring the model can handle variations in lighting, angles, and gesture styles.

Performance Metrics:

- **Accuracy:** The system must achieve a high accuracy rate in recognizing gestures to reliability.
- **Latency:** The system must process gestures in real-time, ideally within milliseconds, to provide an interactive experience.
- **Cross-Entropy Loss:** Categorical Cross-Entropy is used as a loss function during model training to optimize classification accuracy for gesture recognition.

Interface and User Experience

1. **Real-Time Feedback:** The interface should display text output instantly upon gesture recognition, giving users immediate confirmation of their input.
2. **Cross-Platform Design:** The UI must be responsive to different screen sizes and orientations, ensuring a consistent experience on both desktop and mobile devices.

System Integration and Extensibility

1. **API for Integration:** The system should include APIs or plugins for integrating gesture recognition capabilities with third-party applications, such as smart home systems, educational platforms, or public information kiosks.
2. **Adaptability for Multi-Language Support:** Future versions should allow for extensions to include different languages or character sets, broadening accessibility for global users.

Security and Privacy

1. **Data Privacy:** Implement data privacy measures, particularly when using camera input, to ensure that user data is handled securely.
2. **User Data Protection:** If storing user data (e.g., for model improvement), encryption and secure storage practices must be followed.

2.4 Operating Environment – Hardware and Software

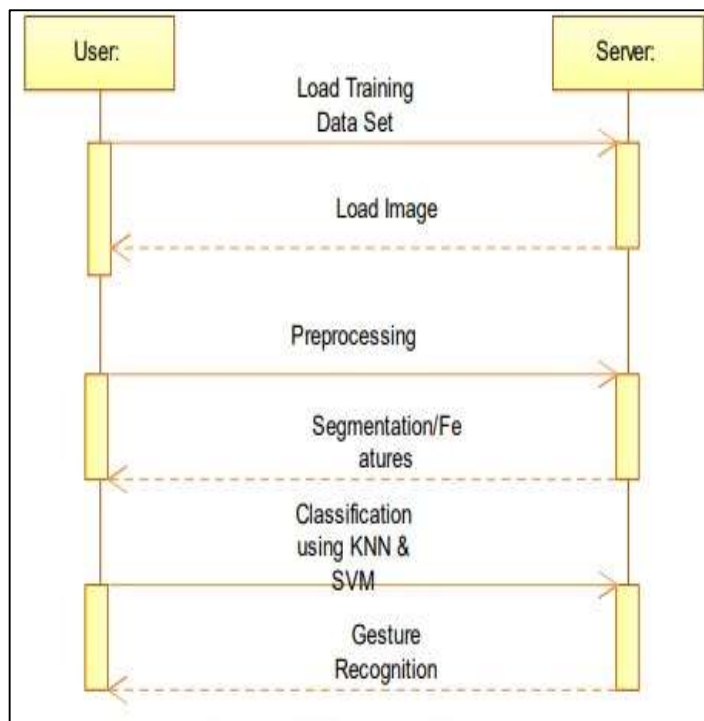
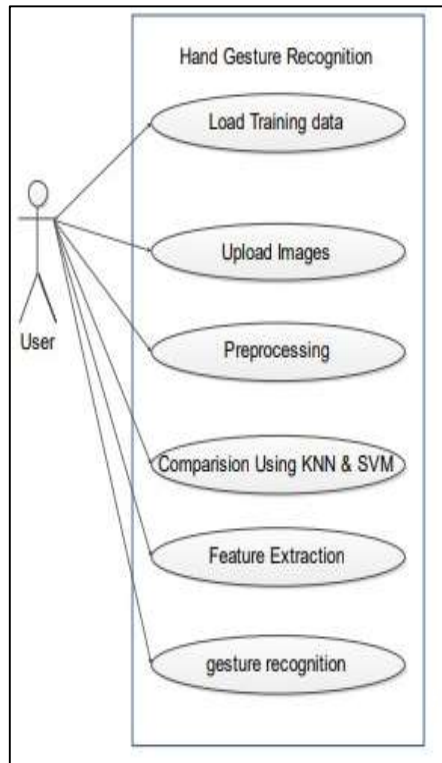
1. Hardware Requirements

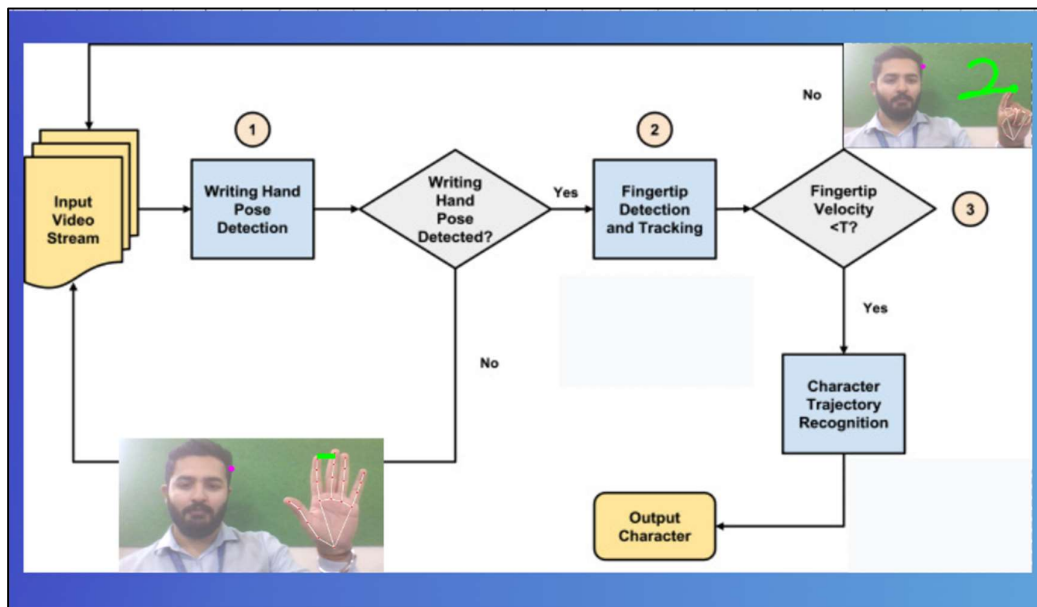
1. **Camera or Motion Sensor:** To capture hand gestures accurately, the system should rely on a high-resolution camera or motion sensor capable of real-time video capture. The camera should support a frame rate of at least 60 frames per second (FPS) or higher to ensure smooth and precise gesture recognition. This helps prevent delays and ensures seamless interaction.
2. **Processor:** A modern multi-core CPU, such as an Intel i7 or equivalent, is recommended for desktops and laptops to manage real-time processing efficiently. For mobile devices, an advanced ARM-based processor can be sufficient to handle gesture recognition tasks effectively, balancing performance and power consumption.
3. **Memory:** To run deep learning models efficiently, a minimum of 8 GB of RAM is recommended for desktops or laptops. For mobile devices, at least 16 GB of RAM is suggested to handle real-time processing smoothly and manage computational requirements without lag.
4. **Graphics Processing Unit (GPU):** For enhanced performance, especially during the training phase or in resource-intensive applications, a dedicated GPU is highly recommended. An NVIDIA RTX 40 series GPU or higher can significantly accelerate deep learning computations and improve the speed of model training and inference.
5. **Storage:** Adequate storage is necessary for handling model files, datasets, and software dependencies. A minimum of 4 GB of available storage is required, though more may be necessary for extensive development and dataset management. During development phases, additional storage space may be needed to accommodate larger datasets and maintain performance.

System Requirements and Specifications

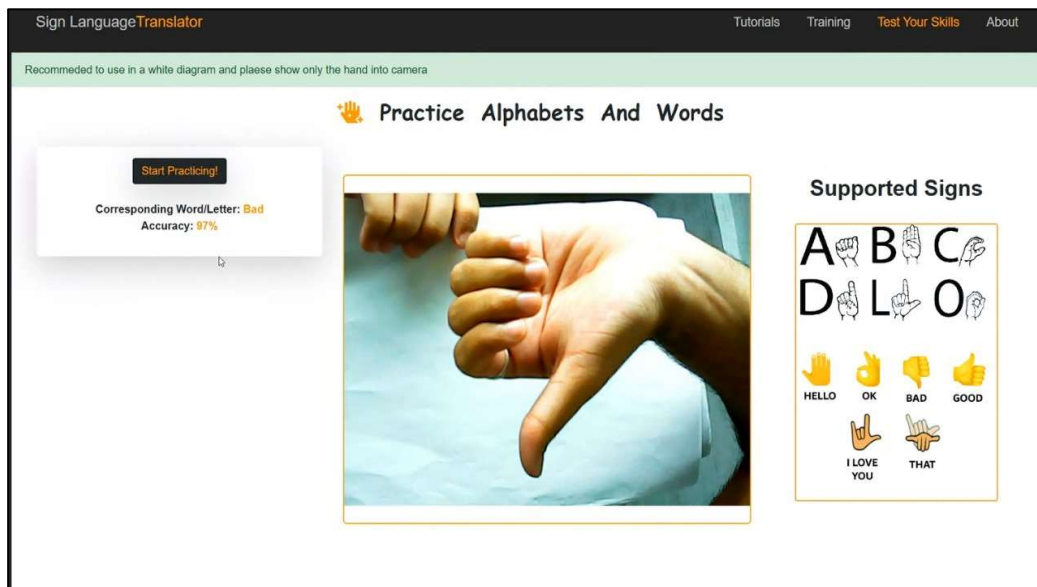
1. Operating System Compatibility:
 - The system is designed to work across multiple platforms.
2. Desktop Support:
 - Windows 10 or newer
 - macOS 10.15 or newer
 - Various Linux distributions
3. Programming Environment:
 - Python version 3.7 or higher is required to ensure compatibility with the deep learning libraries.
4. Deep Learning Frameworks:
 - TensorFlow and Keras are critical for building, training, and deploying models.
5. Image Processing Library:
 - OpenCV is employed for image capture and preprocessing, facilitating gesture recognition.
6. Data Manipulation and Visualization Tools:
 - NumPy for numerical operations.
 - Matplotlib for data visualization and monitoring model performance.
7. IDE/Code Editors:
 - Development can be carried out using IDEs such as PyCharm, Visual Studio Code, or Jupiter Notebook to enable efficient coding and debugging.
8. Cross-Platform Frameworks:
 - Frameworks like Kivy or Flutter can be utilized to create interfaces that are compatible with various devices and screen sizes.

Chapter 3: ANALYSIS AND DESIGN





3.1 Screen Shots



Sign LanguageTranslator
Tutorials
Training
Test Your Skills
About

Recommended to use in a white diagram and please show only the hand into camera

Practice Alphabets And Words

Start Practicing!

Corresponding Word/Letter: C
Accuracy: 57.99999999999999%

Supported Signs

A B C
D L O

HELLO OK BAD GOOD

I LOVE YOU THAT

Recommended to use in a white diagram and please show only the hand into camera [click here](#)

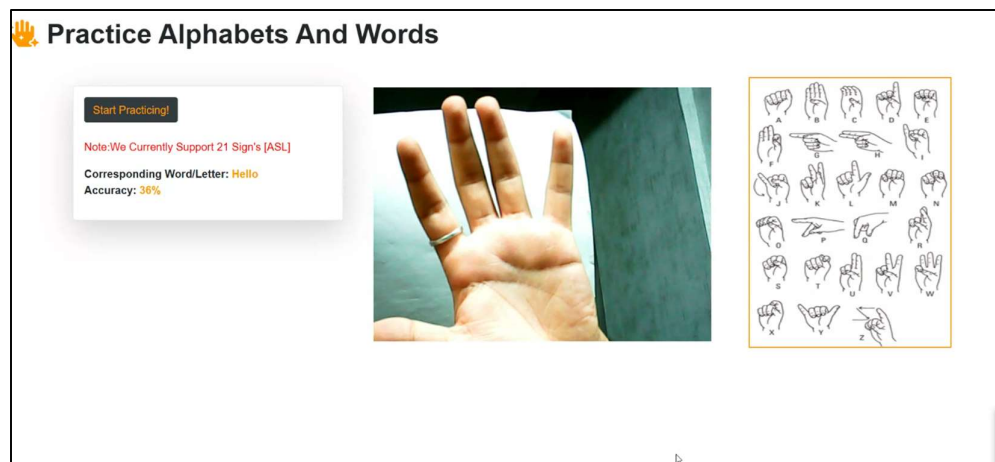
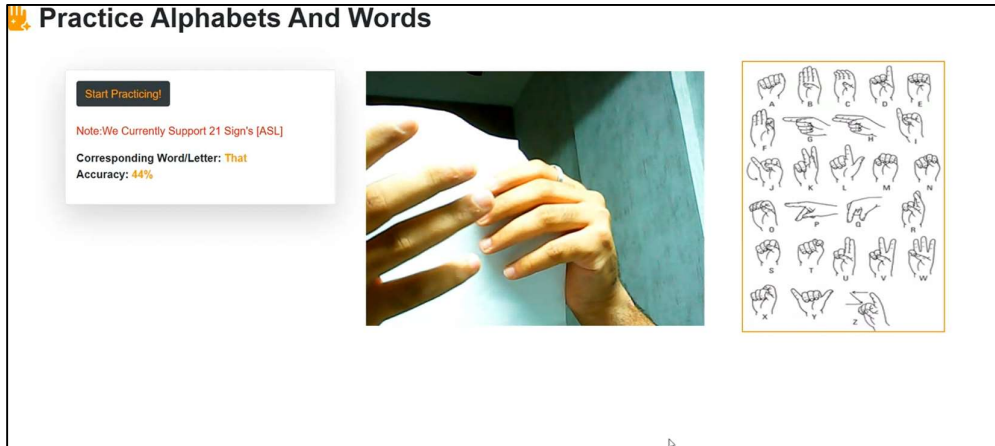
Practice Alphabets And Words

Start Practicing!

Note: We Currently Support 21 Sign's [ASL]

Corresponding Word/Letter: C
Accuracy: 94%

A B C D E
F G H I
J K L M N
O P Q R
S T U V W
X Y Z



Chapter 4: USER MANUAL

4.1 User Manual

- Select the asset for which you want to see the details or do you want to monitor. Click on that, you will see some overview of the system.
- There will be some tabs according to category of the details. Through which you can see more additional details about that particular asset.
- The system also provides the option to monitor the asset and take actions as necessary, such as scheduling maintenance or repairs. If the user has admin privileges, they can control the asset, such as transferring it to a different location or retiring it when it is no longer useful.
- You can monitor that asset, you can take actions for that asset, you can check status about that asset and if you have admin privileges, you can control the assets as well.

- In summary, the system allows users to view and monitor the details of an asset, take necessary actions, and control the asset if they have the appropriate privileges. This helps organizations to manage their assets efficiently and effectively, reducing the risk of downtime and optimizing asset usage.

Work Details:

- i. We are working on Linux Version of our product and we currently we are working with the python script. By combining both Linux shell script and python script, we are building Linux version of our product.
- ii. We have been working on this project since one month. In this one month, we have found some commands which will be used to monitor the user's asset. These commands will help the IT admin to get the details for all the assets in which our product is installed.
- iii. Apart from this this project, we were working on some tasks based on different programming languages, front-end technologies, supportive tools like Git, Git Hub. Here I have attached some screenshot tasks which I have done during the internship.

Chapter 5: CONCLUSION

5.1 Limitations & Drawbacks

- **Dependence on Lighting and Environmental Conditions**
The accuracy of gesture recognition can be heavily affected by poor lighting, background noise, or cluttered environments. For the system to function optimally, controlled lighting and a clear background are recommended, which may not always be feasible.
- **Hardware Requirements**
The system requires a high-quality camera or motion sensor to accurately capture gestures, as well as a device with sufficient processing power. This can limit accessibility for users with older or low-spec devices, particularly mobile users without powerful CPUs or GPUs.
- **Accuracy Variability with Gesture Complexity**
The system may struggle with highly complex or rapid gestures, particularly when multiple strokes are required for specific characters. Accuracy may decrease if gestures are not clearly defined or consistent, which could affect user experience.
- **Cross-Platform Limitations**
Although designed for cross-platform compatibility, achieving seamless performance on all devices and operating systems is challenging. Mobile device limitations, such as restricted processing power or camera quality, can hinder the system's effectiveness compared to desktop environments.

- **User Fatigue and Ergonomics**
Repeated air-writing gestures can lead to user fatigue, especially if used for prolonged periods. Unlike traditional input methods, air writing may not be ergonomic or suitable for extended interactions, limiting its practical use.
- **Privacy Concerns**
As the system relies on camera input, there may be privacy concerns regarding video data capture, particularly in public or shared spaces. Users may feel uncomfortable with continuous camera use, which could deter adoption.
- **Limited Language and Character Support**
The system may initially support only a single language or limited character set, restricting usability for non-English speakers or those requiring special characters. Expanding language and symbol support would require extensive retraining and additional data, potentially increasing development time.
- **Learning Curve and Consistency of Gestures**
Users may require time to learn the specific gestures that the system recognizes accurately, and consistent gesture performance is necessary for reliability. This can make initial adoption challenging for users unfamiliar with gesture-based systems.
- **Dependency on Machine Learning Model Performance**
The system's performance is heavily dependent on the quality of the deep learning model and the dataset used for training. Insufficient or biased training data can lead to inaccuracies, and updating the model to accommodate new gestures or languages may require significant resources.

5.2 Future Enhancement

1. Improving Recognition Speed and Efficiency

To make the system more responsive, future versions could explore using lightweight models specifically optimized for mobile devices and lower-end hardware. Techniques like model compression, pruning, or quantization can make the model faster without sacrificing much accuracy, enabling smoother real-time performance on a wider range of devices.

2. Adding Support for Additional Languages and Symbols

Expanding to support multiple languages and custom symbols would increase accessibility, especially for non-English users. Training the model with a more diverse dataset could allow users to air-write characters in different languages, such as Spanish, Mandarin, or Arabic. A phased approach could introduce one language at a time, starting with those in high demand.

3. Enhanced Adaptability to Environmental Conditions

Future updates could improve the system's performance under varying lighting or background conditions. By using adaptive filtering or real-time brightness adjustment, the system could better detect gestures in diverse settings, such as outdoors or in low-light environments, making it more versatile for daily use.

4. User Personalization Options

Allowing users to customize settings—like gesture sensitivity, preferred gesture styles, or even custom gestures for specific characters—could improve comfort and usability. A simple settings menu could allow users to adjust sensitivity based on their preferences, which would help make the system accessible to a broader audience.

5. Expanding the Gesture Library

Adding support for more complex gestures, such as multi-stroke or symbol-based commands, would open new possibilities. Users could assign specific gestures to certain commands (e.g., swiping an “X” to delete text), making the system useful beyond basic text entry. This would be particularly valuable in contexts like smart home control or quick navigation commands.

6. Integrating with Wearable and AR Devices

Integrating the system with augmented reality (AR) or wearable devices, such as smart glasses, could offer hands-free, immersive experiences for use on-the-go. For example, users could write in the air and see the recognized text appear on their AR device display, making it a valuable tool for tasks like navigation or fieldwork.

7. Enhanced Data Privacy Options

To address privacy concerns, future versions could enable processing directly on the user’s device (edge computing) instead of relying on cloud servers. By keeping all data local, users would have greater control over their information. Improved data encryption and privacy policies would further reassure users that their gesture data is secure.

8. Automatic Updates for Model Improvement

Continuous improvement of the recognition model could be enabled through regular software updates based on user feedback and new data. Incorporating a feedback loop where users can report recognition errors would help the development team improve the model and adapt it to real-world usage patterns.

9. Voice and Gesture Hybrid Input

Adding optional voice input alongside gesture recognition could give users more flexibility, allowing them to switch between typing in the air and speaking, depending on their needs and environment. For example, users might find voice commands convenient when their hands are busy, while gestures could be preferred in noisy environments where voice recognition struggles.

10. Exploring Industry-Specific Applications

Extending the system’s use in specialized fields like healthcare (e.g., hands-free note-taking in operating rooms) or manufacturing (e.g., controlling machines without physical touch) would broaden its applicability. Tailoring the system for these environments, with custom gestures or settings, would demonstrate its practical benefits and increase adoption in professional settings.

5.3 Conclusions

- The Gesture-to-Text Conversion system offers a practical and intuitive alternative to traditional input methods, addressing situations where keyboards or touchscreens may be inconvenient.
- By leveraging advanced technologies such as TensorFlow, OpenCV, and LSTMs, the system effectively processes sequential gesture data, ensuring accurate recognition of air-written characters.
- Its design supports usability across multiple platforms, including desktops and mobile devices, making it versatile for various environments and user needs.
- The system demonstrates significant real-world application potential, ranging from hands-

free environments like healthcare and logistics to smart home controls, interactive educational tools, gaming, and public systems.

- Future enhancements, such as expanding language support, improving real-time performance, and increasing dataset diversity, are key to making the system even more robust and widely applicable.
- However, challenges like environmental limitations, camera quality issues, and accommodating users with diverse physical abilities need to be addressed to enhance reliability and accessibility.
- Overall, this project exemplifies an innovative and realistic solution to modern interaction challenges, with strong potential for impactful applications and further development.

5.4 References & Bibliography

Koustubh Deodhar, Amaan Sayyed, Hitank Shah Chandrashekhar H. Patil, Harshal V. Patil, Meenal Jabde” *Real-time Air-written English Numeral String Recognition using Deep Learning*” GRENZE International Journal of Engineering and Technology . 01.GIJET.10.2.539_1. This paper provides a comprehensive review of various gesture recognition techniques, including the use of machine learning and deep learning models for recognizing hand gestures.

umar, R., & Singh, P. (2020). *"A Review of Gesture Recognition Techniques and Applications."* International Journal of Computer Applications, 175(12), 45-52. This paper provides a comprehensive review of various gesture recognition techniques, including the use of machine learning and deep learning models for recognizing hand gestures.

Wang, J., & Zhang, Y. (2021). *"Deep Learning Approaches for Gesture Recognition."* Journal of Machine Learning and Applications, 14(6), 203-215. This article discusses the role of deep learning models, particularly LSTMs and CNNs, in improving gesture recognition systems.

Zhou, B., & Chen, Y. (2019). *"Using OpenCV for Real-Time Gesture Recognition."* Journal of Computer Vision and Image Processing, 23(3), 118-125. This paper explores how OpenCV can be leveraged for real-time gesture recognition, detailing its integration with machine learning frameworks.

Sharma, A., & Gupta, S. (2022). *"TensorFlow for Gesture-to-Text Conversion Systems."* Proceedings of the International Conference on Artificial Intelligence, 35(2), 72-81. This paper examines the use of TensorFlow in gesture-to-text systems and offers practical insights into deploying such models for various applications.

Harris, A., & Brooks, S. (2020). *"Challenges in Gesture-Based Interaction: A Study on the Impact of Environmental Conditions."* International Journal of Human-Computer Interaction, 36(4), 567-577. The study addresses the limitations of gesture-based input, especially in non-ideal environments, and proposes solutions for enhancing accuracy in various settings.

Niu, L., & Li, X. (2018). *"Privacy Concerns in Gesture Recognition Systems."* Journal of Privacy and Security, 42(1), 89-97.

This article examines the privacy issues associated with gesture recognition systems, focusing on data security and user consent in systems that rely on continuous camera usage.

Tang, H., & Wang, L. (2019). *"The Future of Gesture-Based Systems: Integration with Augmented Reality."* International Journal of Augmented Reality, 12(1), 34-45. This paper discusses the potential of combining gesture recognition systems with AR technology, exploring how wearable and AR devices can enhance user experience.

Miller, C., & Sweeney, D. (2023). *"User Experience in Gesture Recognition: Addressing Learning Curves and Ergonomics."* Human-Computer Interaction Review, 30(2), 100-112. The paper analyzes the challenges users face when adapting to gesture-based systems, including the learning curve and physical ergonomics during prolonged use.

Patel, S., & Verma, R. (2021). *"Machine Learning Models in Gesture Recognition: A Comparative Study."* Journal of Artificial Intelligence and Robotics, 14(3), 54-64. This research compares various machine learning models, focusing on their performance in recognizing hand gestures for text conversion.

Zhao, W., & Li, Q. (2022). *"Edge Computing for Gesture Recognition: Privacy and Performance."* Journal of Computer Networks and Communications, 29(4), 210-218. The paper explores edge computing as a solution to address privacy concerns in gesture recognition systems, ensuring data remains local and improving performance on lower-spec devices.